

Controllo della Concorrenza

In sistemi di calcolo monoprocesso (con una sola CPU) i programmi vengono eseguiti in maniera concorrente, in un modo che viene detto interleaved (interfogliato), in cui la CPU può:

- Eseguire alcune istruzioni di un programma
- Sospendere quel programma
- Eseguire istruzioni di altri programmi
- Ritornare ad eseguire istruzioni del primo

Tale "esecuzione concorrente" permette un uso efficiente della CPU. In un DBMS la principale risorsa a cui i programmi accedono in modo concorrente è la base di dati.

Se sulla BD vengono effettuate solo letture (la BD non viene mai modificata), l'accesso concorrente non crea problemi. Se invece vengono effettuate anche scritture (la BD viene modificata), l'accesso concorrente può creare problemi di inconsistenza di dati, e deve essere controllato.

Transazioni

def. Una transazione è esecuzione di una parte di un programma che rappresenta un'unità logica di accesso o modifica del contenuto della base di dati

Riguardo le transazioni facciamo riferimento all'acronimo inglese ACID (Atomicità, Consistenza, Isolamento e Durabilità) che indica le proprietà logiche che devono avere le transazioni.

- **Atomicità:** la transazione è indivisibile nella sua esecuzione e la sua esecuzione deve essere o totale o nulla, non sono ammesse esecuzioni parziali
- **Consistenza:** quando inizia una transazione il database si trova in uno stato consistente e quando la transazione termina il database deve essere in un altro stato consistente. Cioè non deve violare eventuali vincoli di integrità.
- **Isolamento:** ogni transazione deve essere eseguita in modo isolato e indipendente, non deve interferire con le altre transazioni in esecuzione;
- **Durabilità:** i cambiamenti apportati non dovranno essere più persi, ad esempio fra il momento in cui la base di dati si impegna a scrivere le modifiche e quello in cui li scrive effettivamente

Item e Granularità

Item: unità a cui l'accesso è controllato. In parole povere un item è un attributo.

Granularità: Le dimensioni degli item usati da un sistema sono dette la sua granularità. Tale dimensione è deve essere definita in base all'uso che viene fatto della base di dati in modo tale che in media una transazione acceda a pochi item. Una granularità grande permette una gestione efficiente della concorrenza. Una granularità piccola può sovraccaricare il sistema, ma aumenta il livello di concorrenza (poiché possiamo eseguire più transazioni insieme)

Schedule

Lo Schedule è il piano di esecuzione di un insieme T di transazioni, cioè l'ordine in cui le operazioni delle transazioni in T verranno eseguite. Esso deve conservare l'ordine che le operazioni hanno all'interno delle singole transazioni.

Schedule Seriale

In particolare noi ci interesseremo degli Schedule seriali, ovvero le schedule ottenuti permutando le transazioni in T. Quindi:

def. Uno Schedule Seriale corrisponde ad una esecuzione sequenziale (non interfogliata) delle transazioni

Schedule Serializzabile

Def. Uno Schedule Serializzabile è un qualsiasi schedule equivalente ad uno Schedule Seriale

Problemi

Quando una transazione richiede un item esso viene prelevato dalla memoria secondaria ed inserito in uno spazio della memoria principale PRIVATO, cioè riservato alla singola transazione. Questo comporta che se due transazioni richiedono lo stesso dato e poi lo modificano, esso sarà modificato in modo diverso nelle due aree di memoria (causando incongruenza).

Esempio: Consideriamo le due transazioni:

- T1 può essere interpretata come il trasferimento di una somma di denaro N dal conto corrente X al conto corrente Y
- T2 può essere interpretata come l'accredito sul conto corrente X di una somma di denaro M

Ghost Update – Perdita di aggiornamento

Consideriamo il seguente schedule di T1 e T2.

T_1	T_2
$read(X)$ $X:=X-N$	$read(X)$ $X:=X+M$
$write(X)$ $read(Y)$	$write(X)$
$Y:=Y+N$ $write(Y)$	

Se il valore iniziale di X è X_0 , al termine dell'esecuzione dello schedule il valore di X è X_0+M invece di X_0-N+M , e dunque l'aggiornamento di X prodotto da T1 viene perso.

Esempio:

- La transazione A legge un dato.
- La transazione B modifica lo stesso dato e lo conferma.
- La transazione A, senza rileggere il dato, esegue un'operazione basata sul valore precedente, causando inconsistenze

Dato Sporco

Consideriamo il seguente schedule di T1 e T2:

T_1	T_2
$read(X)$ $X := X - N$ $write(X)$	$read(X)$ $X := X + M$
$read(Y)$ T_1 fallisce	$write(X)$

Se il valore iniziale di X è X0 al termine dell'esecuzione dello schedule il valore di X è X0-N+M invece di X0+M. Il valore di X letto da T2 è un dato sporco (poiché è stato scritto prima di controllare che la transazione andasse a buon fine).

Aggregato NON Corretto

Consideriamo il seguente schedule di T1 e T3:

T_1	T_3
$read(X)$ $X := X - N$ $write(X)$	$somma := 0$
	$read(X)$ $somma := somma + X$
	$read(Y)$ $somma := somma + Y$
$read(Y)$ $Y := Y + N$ $write(Y)$	

Se il valore iniziale di X è X0 e il valore iniziale di Y è Y0 al termine dell'esecuzione dello schedule il valore di somma è X0-N+Y0 invece di X0+Y0. Il valore di somma è un dato aggregato non corretto (Abbiamo preso X aggiornato ma Y no). **Esempio:**

- La transazione A inizia a calcolare la somma di una colonna.
- La transazione B inserisce o modifica alcuni valori nella stessa colonna.
- La somma calcolata da A potrebbe non includere le modifiche di B o includerle parzialmente, portando a un risultato errato

Schedule Corretti

Perché nei tre i casi visti siamo portati a considerare gli schedule non corretti? Perché i valori prodotti non sono quelli che si avrebbero se le due transazioni fossero eseguite nel modo “naturale” cioè sequenzialmente.

Def. Tutti gli schedule seriali sono corretti

Schedule Equivalenti

Uno schedule non seriale è corretto se è serializzabile, cioè se è “equivalente” ad uno schedule seriale. Ma che significa equivalenti?

Def. Due schedule sono equivalenti se per ogni dato modificato producono valori uguali, con la stessa sequenza di operazioni

Esempio: I seguenti Schedule NON sono equivalenti anche se danno lo stesso risultato su X
(Si noti che non sono equivalenti, ma sono entrambi seriali ... quindi corretti!)

T_1	T_2	T_1	T_2
$read(X)$			$read(X)$
$X:=X+N$			$X:=X-M$
$write(X)$			$write(X)$
	$read(X)$	$read(X)$	
	$X:=X-M$	$X:=X+N$	
	$write(X)$	$write(X)$	

Testare la Serializzabilità?

Testare la serializzabilità è praticamente impossibile a causa di alcuni problemi «pratici». Ad esempio:

- È difficile stabilire quando uno schedule comincia e quando finisce
- È praticamente impossibile determinare in anticipo in quale ordine le operazioni verranno eseguite
- Se prima si eseguono le operazioni e poi si testa la serializzabilità, i suoi effetti devono essere annullati se lo schedule risulta non serializzabile

Come Risolviamo?

L'approccio seguito nei sistemi è quello di **garantire la serializzabilità** degli schedule, eliminando la necessità di testare la serializzabilità. Per fare ciò si utilizzano diversi metodi, quelli che studiamo noi sono:

- **Protocolli:** cioè imporre delle regole alle transazioni in modo da garantire la serializzabilità
- **Timestamp delle transazioni:** cioè degli identificatori delle transazioni che vengono generati dal sistema e in base ai quali le operazioni delle transazioni possono essere ordinate in modo da garantire la serializzabilità.

Locking

Un Lock garantisce il privilegio di accesso ad un singolo item realizzato mediante una variabile associata (**variabile lucchetto**) il cui valore descrive lo stato dell'item rispetto alle operazioni che possono essere effettuate su di esso. Il locking nella sua forma più semplice consente 2 operazioni:

- **Operazione di Locking:** Una transazione richiede di accedere all'item, se il valore della variabile è unlocked la transazione può accedere e alla variabile viene assegnato il valore locked (altrimenti no)
- **operazione di Un-locking:** La transazione che ha finito di usare l'item la rilascia, assegnando alla variabile il valore unlocked

Schedule Legale

def. Uno schedule è detto legale se una transazione effettua un locking ogni volta che deve leggere o scrivere un item (e successivamente ciascuna transazione rilascia il lock ottenuto)

Lock Binario

Un lock binario può assumere solo due valori: locked o unlocked. Le transazioni fanno uso di due operazioni

- **Lock(X)** per richiedere l'accesso all'item X. (Implica la lettura di X)
- **Unlock(X)** per rilasciare l'item X consentendone l'accesso ad altre transazioni. (Implica la scrittura di X)

L'insieme degli item letti e quello degli item scritti da una transazione coincidono

Il Lock Binario risolve il primo problema, ovvero quello del Ghost Update (o perdita di aggiornamento). Infatti, riprendono l'esempio con:

- T1 = trasferimento di una somma di denaro N dal conto corrente X al conto corrente Y
- T2 = accredito sul conto corrente X di una somma di denaro M

Prima si aveva che, con lo schedule di seguito, l'aggiornamneto di X prodotto da T1 andava perso. Utilizzando il Lock Binario otteniamo invece:

T_1	T_2
$read(X)$ $X := X - N$	$read(X)$ $X := X + M$
$write(X)$ $read(Y)$	$write(X)$
$Y := Y + N$ $write(Y)$	

➔

T_1	T_2
$lock(X)$ $read(X)$ $X := X - N$ $write(X)$ $unlock(X)$	$lock(X)$ $read(X)$ $X := X + M$ $write(X)$ $unlock(X)$
$lock(Y)$ $read(Y)$ $Y := Y + N$ $write(Y)$ $unlock(Y)$	

Notare che questo è uno Schedule Legale di T1 e T2

Riformulazione

Arrivati a questo punto vedremo che l'equivalenza degli schedule dipende dal protocollo di locking adottato. In particolare, nel caso del Locking Binario, effettuiamo una riformulazione del concetto di transazione così da rendere le idee più chiare e semplici.

def. Una transazione è una sequenza di operazioni di lock e unlock, in cui:

- Ogni $\text{lock}(X)$ implica la lettura di X
- Ogni $\text{unlock}(X)$ implica la scrittura di X

In corrispondenza di ogni Unlock (scrittura) viene associato un nuovo valore all'item tramite una funzione, che è associata in modo univoco ad ogni coppia lock-unlock e che ha come parametri TUTTI gli item letti (cioè coinvolti in qualche Lock precedente)

T_1
$\text{lock}(X)$
$\text{unlock}(X) f_1(X)$
$\text{lock}(Y)$
$\text{unlock}(Y) f_2(X, Y)$

Due schedule sono equivalenti se le formule che danno i valori finali per CIASCUN item sono le stesse.

Uno schedule è serializzabile se è equivalente ad uno schedule seriale.

Esempio:

Consideriamo le due transazioni

T_1
$\text{lock}(X)$
$\text{unlock}(X) f_1(X)$
$\text{lock}(Y)$
$\text{unlock}(Y) f_2(X, Y)$

T_2
$\text{lock}(Y)$
$\text{unlock}(Y) f_3(Y)$
$\text{lock}(X)$
$\text{unlock}(X) f_4(X, Y)$

e lo schedule

	T_1	T_2	
legge X_0	$lock(X)$		
scrive $f_1(X_0)$	$unlock(X)$		
		$lock(Y)$	legge Y_0
		$unlock(Y)$	scrive $f_3(Y_0)$
legge $f_3(Y_0)$	$lock(Y)$		
scrive $f_2(X_0, f_3(Y_0))$	$unlock(Y)$		
		$lock(X)$	legge $f_1(X_0)$
		$unlock(X)$	scrive $f_4(f_1(X_0), Y_0)$
X_0 valore iniziale di X Y_0 valore iniziale di Y	$f_4(f_1(X_0), Y_0)$ valore finale di X		

Per verificare che tale Schedule sia Serializzabile ci è sufficiente prendere uno schedule seriale, calcolarne il valore finale di X e vedere se esso è uguale a quello prodotto dal nostro schedule. Proviamo con:

Schedule Seriale T_2, T_1

	T_1	T_2	
		$lock(Y)$	legge Y_0
		$unlock(Y)$	scrive $f_3(Y_0)$
		$lock(X)$	legge X_0
		$unlock(X)$	scrive $f_4(X_0, Y_0)$
legge $f_4(X_0, Y_0)$	$lock(X)$		
scrive $f_1(f_4(X_0, Y_0))$	$unlock(X)$		
legge $f_3(Y_0)$	$lock(Y)$		
scrive $f_2(f_4(X_0, Y_0), f_3(Y_0))$	$unlock(Y)$		
	$f_1(f_4(X_0, Y_0))$ valore finale di X		

Poiché il risultato è diverso abbiamo che, PER ORA, lo Schedule non è serializzabile. Proviamo a verificare se è equivalente allo Schedule Seriale T_2, T_1 :

Consideriamo lo
schedule seriale T_2, T_1

legge $f_4(X_0, Y_0)$
 scrive $f_1(f_4(X_0, Y_0))$
 legge $f_3(Y_0)$
 scrive $f_2(f_4(X_0, Y_0), f_3(Y_0))$

T_1	T_2
	$lock(Y)$
	$unlock(Y)$
	$lock(X)$
	$unlock(X)$
$lock(X)$	
$unlock(X)$	
$lock(Y)$	
$unlock(Y)$	

legge Y_0
 scrive $f_3(Y_0)$
 legge X_0
 scrive $f_4(X_0, Y_0)$

$f_1(f_4(X_0, Y_0))$ valore finale di X

prodotto dallo schedi

Pertanto lo schedule non è serializzabile in quanto produce per X un valore finale diverso sia da quello prodotto dallo schedule seriale T_1, T_2 che da quello dello schedule seriale T_2, T_1 . E stessa cosa varrebbe per Y. Infatti basta che le formule siano diverse anche per un solo item per concludere che gli schedule non sono equivalenti. Questo comporta che:

- **Per verificare che uno schedule NON è serializzabile** possiamo fermarci appena troviamo un item le cui formule finali sono diverse da quelle di OGNI schedule seriale
- **Per verificare invece che uno schedule è Serializzabile** occorre verificare che le formule finali di TUTTI gli item coincidono con quelle di uno schedule seriale

Alogirtmo per Testare la Serializzabilità

Concludiamo quindi che

Uno schedule è serializzabile se esiste uno schedule seriale tale che, **per ogni item**, l'ordine in cui le varie transazioni fanno un lock su quell'item coincide con quello dello schedule seriale

BEGIN

Passo 1: Creare un grafo di serializzazione G, in cui:

- **Nodi:** sono Transizioni ($T_1, T_2, \dots$)
- **Arch:** $T_i \rightarrow T_j \Leftrightarrow T_i$ fa unlock su una risorsa e T_j è IL successivo a fare lock su di essa

Passo 2: Se G ha un ciclo allora lo Schedule NON è serializzabile;

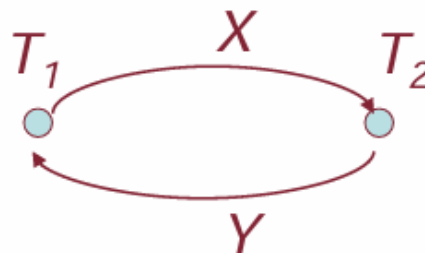
Altrimenti applichiamo a G l'ordinamento topologico così da ottenere uno schedule seriale S' equivalente ad S. A questo punto concludiamo che **S è serializzabile** $\Leftrightarrow$ **il suo grafo di serializzazione è aciclico**

END

Esempio1:

T_1	T_2
$lock(X)$ $unlock(X)$	$lock(Y)$ $unlock(Y)$
$lock(Y)$ $unlock(Y)$	$lock(X)$ $unlock(X)$

T1 fa unlock su X e T2 lo locka subito dopo



Esempio2:

T_1	T_2
$lock(X)$ $unlock(X)$	$lock(X)$ $unlock(X)$
$lock(Y)$ $unlock(Y)$	$lock(Y)$ $unlock(Y)$

T1 fa unlock su X e T2 lo locka subito dopo

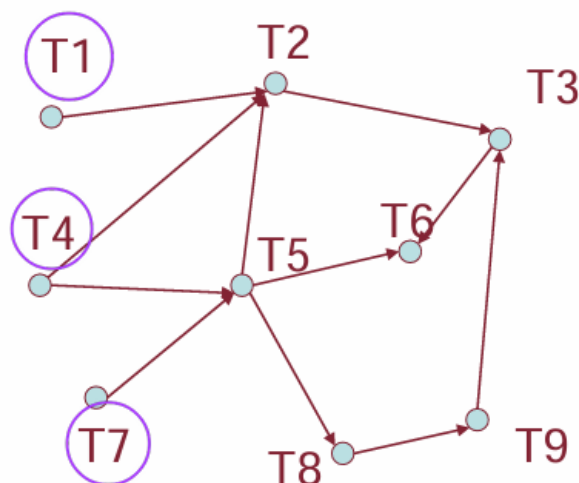


T1 fa unlock anche su Y e T2 lo locka subito dopo

Ordinamento Topologico

Si ottiene eliminando ricorsivamente un nodo che non ha archi entranti, insieme ai suoi archi uscenti.

ATTENZIONE! un grafo può ammettere più ordinamenti topologici



In questo caso T1, T4 e T7 sarebbero i primi ad essere eliminati.

Esempio:

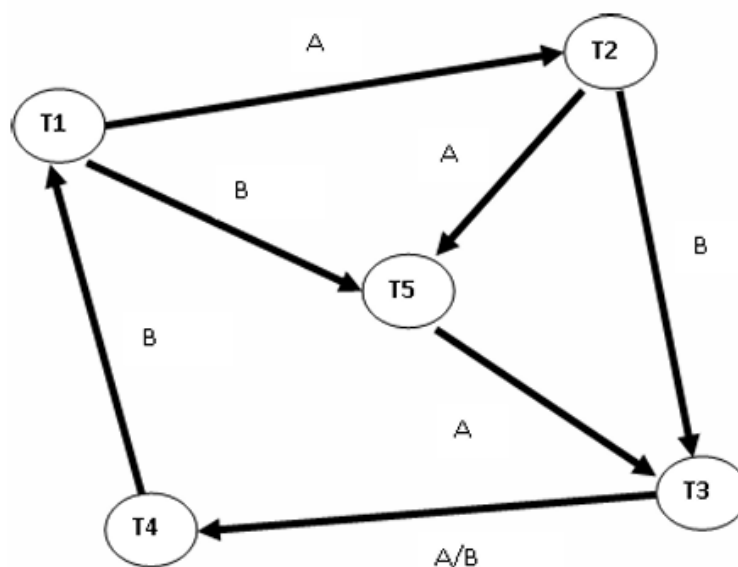
Prendiamo questo schedule di 5 transazioni

T1	T2	T3	T4	T5
LOCK A				
	LOCK B			
UNLOCK A				
	UNLOCK B			
	LOCK A			
		LOCK B		
	UNLOCK A			
				LOCK A
		UNLOCK B		
		LOCK A		UNLOCK A
			LOCK B	
		UNLOCK A		
			UNLOCK B	
LOCK B			LOCK A	
			UNLOCK A	
UNLOCK B				
				LOCK B
				UNLOCK B

Il grafo di serializzazione è:



O, rappresentato meglio per evidenziare il ciclo:



Il grafo presenta il ciclo T1-T2-T3-T4. Possiamo quindi concludere che lo schedule dato non è serializzabile.

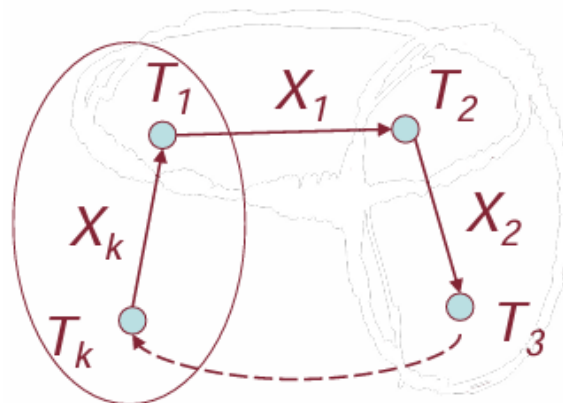
Protocollo di Locking a Due Fasi

def. Una transazione è "a due fasi", se prima effettua tutte le operazioni di lock (fase di locking) e poi tutte le operazioni di unlock (fase di unlocking)

ATTENZIONE! Da non confondere con il lock a due valori! Il fatto di essere a due fasi è una caratteristica in più, ma ci sono protocolli a due fasi ... e tre valori.

TEOREMA: Se ogni transazione in T è a due fasi allora ogni schedule di T è serializzabile

Dim: Supponiamo per assurdo che ogni transazione in S sia a due fasi e che contemporaneamente nel grafo ci sia un ciclo. In questo caso avremmo il grafo:



Il tratteggiato serve solo a dire che lì in mezzo ci sono tante altre transazioni a 2 fasi

Ma questo grafo ha senso se e solo se S è:

S	
...	
T_1	unlock(X_1)
...	
T_2	lock(X_1)
...	
T_2	unlock(X_2)
...	
T_3	lock(X_2)
...	
T_k	unlock(X_k)
...	
T_1	lock(X_k)
...	

In cui però T1 NON è a due fasi (abbiamo un lock dopo un unlock) => **CONTRADDIZIONE**

ATTENZIONE! Solo se TUTTE le transazioni sono a due fasi possiamo avere la certezza che ogni schedule è serializzabile

Risolve il Problema dell'Aggregato

Il protocollo 2PL garantisce che le transazioni vengano eseguite in un ordine serializzabile, evitando interferenze tra transazioni concorrenti. Ecco come risolve il problema dell'aggregato non corretto:

1. **Isolamento delle Transazioni:** Durante la fase di Locking, la transazione che esegue l'aggregazione acquisisce lock su tutti i dati necessari. Questi lock impediscono ad altre transazioni di modificare i dati fino a quando la transazione non li rilascia (Fase di Unlocking).
2. **Consistenza dei Dati:** Poiché i dati sono bloccati durante l'aggregazione, nessuna transazione concorrente può modificarli. Questo garantisce che il risultato dell'aggregazione sia basato su uno stato consistente e completo dei dati.

Abort di una Transazione

L'abort di una transazione è un evento in cui una transazione viene interrotta prima del suo completamento, annullando tutte le operazioni eseguite fino a quel momento e ripristinando il database allo stato precedente all'inizio della transazione. Un abort può verificarsi quando:

1. La transazione esegue un'operazione non corretta (divisione per 0, accesso non consentito)
2. Lo scheduler rileva un deadlock
3. Lo scheduler fa abortire la transazione per garantire la serializzabilità (Timestamp)
4. Si verifica un malfunzionamento hardware o software

Il **punto di commit** di una transazione è il punto in cui la transazione ha ottenuto tutti i lock che gli sono necessari, ha effettuato tutti i calcoli nell'area di lavoro e quindi non può più essere abortita a causa di 1-3

Risolvere il Problema dei Dati Sporchi

Secondo questa definizione possiamo riformulare il problema dei Dati Sporchi, e dire che

“I **Dati Sporchi** sono tutti quei dati scritti da una transazione prima che abbia raggiunto il punto di commit”

Tuttavia con i protocolli visti fino ad ora non siamo ancora in grado di risolvere questo problema. Di fatti:

- **L'uso di lock risolve il problema dell'aggiornamento perso...** ma non quello della lettura di un dato sporco
- **Il protocollo di locking a due fasi risolve il problema dell'aggregato non corretto...** ma non quello della lettura di un dato sporco

Per risolvere il problema della lettura di dati sporchi occorre che le transazioni obbediscano a regole più restrittive del protocollo di locking a due fasi, introduciamo quindi il protocollo di Locking a Due Fasi Stretto.

Protocollo a due Fasi Stretto

Def. Una transazione soddisfa il protocollo di locking a due fasi stretto se:

1. **Non scrive sulla base di dati fino a quando non ha raggiunto il suo punto di commit:** In questo modo se una transazione è abortita allora non ha modificato nessun item nella base di dati
2. **Non rilascia un lock finché non ha finito di scrivere sulla base di dati.** Impedisce che altre transazioni possano leggere dati ancora non modificati.

Deadlock e Livelock

Deadlock: Verificare, Risolvere ed Evitare

Def. Il **deadlock** (o **stallo**) si verifica quando due o più transazioni si bloccano reciprocamente, aspettando che l'altra rilasci una risorsa, creando una situazione di attesa infinita.

Per verificare il sussistere di una situazione di stallo si mantiene il **Grafo di Attesa** nodi in cui:

- **Nodi:** sono le transazioni
- **Archi:** $T_i \rightarrow T_j \Leftrightarrow T_i$ è in attesa di ottenere un lock su un item sul quale T_j mantiene un lock.

Chiaramente, se in tale grafo c'è un ciclo si sta verificando una situazione di stallo che coinvolge le transazioni nel ciclo. Per risolvere il sussistere di una situazione di stallo, una transazione nel ciclo viene **"rolled-back"**:

- a) La transazione è abortita
- b) I suoi effetti sulla base di dati vengono annullati ripristinando i valori dei dati precedenti l'inizio della sua esecuzione
- c) Tutti i lock mantenuti dalla transazione vengono rilasciati
- d) Successivamente viene fatta ripartire.

Tuttavia, prima di risolvere i deadlock, si cerca di evitare che essi si verifichino adottando opportuni protocolli. Ad esempio si impone alle transazioni di richiedere i lock in un determinato ordine tale che non ci possono essere cicli nel grafo di attesa (e quindi non si può verificare un deadlock).

Roll-back a cascata

Quando una transazione T viene abortita devono essere annullati gli effetti sulla base di dati prodotti da T e da qualsiasi transazione che abbia letto quei dati sporchi, causando quello che viene detto Roll-back a Cascata

Livelock

Def. Un **livelock** si verifica quando due o più transazioni continuano a cambiare stato in risposta alle azioni dell'altro, ma nessuno riesce a progredire.

Esempio: È come se due persone si incrociassero in un corridoio stretto e continuassero a spostarsi nella stessa direzione per farsi spazio, bloccandosi a vicenda all'infinito.

Il problema dell'attesa indefinita, può essere risolto:

- **Con una strategia First Come-First Served (FCFS).** Questo approccio è semplice ed equo, poiché garantisce che nessuna transazione venga ignorata indefinitamente.
- **Adottando un sistema a priorità,** in cui la priorità di una transazione aumenta proporzionalmente al tempo in cui rimane in attesa. Questo meccanismo garantisce che le transazioni in attesa da più tempo vengano servite prima, evitando che rimangano bloccate indefinitamente.

Classificazione dei protocolli

I protocolli si dividono principalmente in due categorie:

- **Protocolli Conservativi:** cercano di evitare le situazioni di stallo
- **Protocolli Aggressivi:** cercano di processare le transazioni il più rapidamente possibile anche se ciò può portare a situazioni di stallo

Protocolli Conservativi

Per evitare il verificarsi sia del deadlock che del livelock possiamo imporre che una transazione riceva i lock richiesti se e solo se:

- Tutti i lock sono disponibili (evita il deadlock)
- Nessuna transazione che precede T è in attesa di un lock richiesto da T (evita il livelock)

SVANTAGGI: l'esecuzione di una transazione può essere ritardata; una transazione è costretta a richiedere un lock su ogni item che potrebbe essergli necessario anche se poi di fatto non lo utilizza.

Protocolli aggressivi

Versione più aggressiva: una transazione deve richiedere un lock su un item immediatamente prima di leggerlo o scriverlo. Può verificarsi un deadlock.

Quale Scegliere?

Se la probabilità che due transazioni richiedano un lock su uno stesso item è:

- **Alta:** conviene un protocollo conservativo in quanto evita al sistema il sovraccarico dovuto alla gestione dei deadlock
- **Bassa:** conviene un protocollo aggressivo, in quanto evita al sistema il sovraccarico dovuto alla gestione dei lock

Timestamp

Def. Il Timestamp identifica una transazione ed è assegnato dallo scheduler quando questa ha inizio. In generale il Timestamp può essere il valore di un contatore oppure l'ora di inizio della transazione

Il timestamp indica l'ordine temporale in cui le transazioni sono iniziate. Se una transazione T_1 ha un timestamp inferiore a T_2 , significa che T_1 è iniziata prima di T_2 . Da ciò deriviamo che uno schedule è **serializzabile** se è equivalente a un'esecuzione seriale delle transazioni in ordine di timestamp.

Esempio:

T_1	T_2
$read(Y)$	$read(Y)$
$X := Y + 10$	$X := Y + 5$
$write(X)$	$write(X)$

$TS(T_1) = 110$ $TS(T_2) = 100$

Quindi uno schedule è serializzabile se è equivalente
allo schedule seriale T_2T_1

Read/Write Timestamp

A ciascun item X vengono associati due timestamp:

- **Read Timestamp:** il più grande fra tutti i timestamp di transazioni che hanno letto con successo X
- **Write Timestamp:** il più grande fra tutti i timestamp di transazioni che hanno scritto con successo X

Per controllare la Serializzabilità è necessario che ogni volta che una transazione T cerca di eseguire un $read(X)$ o un $write(X)$, si confronti il timestamp di T con il read e write timestamp di X, per assicurarsi che l'ordine basato sui timestamp non sia violato.

Algoritmo – Verifica della Serializzabilità

T cerca di eseguire una write(X):

1. **Se $read_TS(X) > TS(T)$:** Significa che una transazione più recente ha già letto X. Se T scrivesse X, violerebbe l'ordine temporale, perché la transazione più recente avrebbe letto un valore obsoleto. Quindi, T viene abortita (rollback) per mantenere la consistenza.
2. **Se $write_TS(X) > TS(T)$:** Se siamo qui significa che nessuna transazione più giovane ha ancora letto i dati (altrimenti ci fermavamo al punto precedente dell'algoritmo) MA una transazione più recente ha già scritto X. Quindi T non può sovrascrivere X, perché ciò invaliderebbe le modifiche della transazione più recente. L'operazione di scrittura viene ignorata.
3. **Se nessuna delle condizioni precedenti è soddisfatta: write(X) viene eseguita**
(e di conseguenza il write timestamp è sovrascritto --> $write_TS(X) := TS(T)$)

Esempio:

T_1	T_2	$write_TS(X)$	$read_TS(Y)$
	$read(Y)$		100
$read(Y)$			110
	$X := Y + 5$		
$X := Y + 10$			
$write(X)$		110	
	$write(X)$	$write_TS(X) > TS(T_2)$ $write(X)$ non viene eseguita	

T cerca di eseguire una read(X):

1. **Se $write_TS(X) > TS(T)$:** T non può leggere X, perché il valore corrente è stato prodotto da una transazione successiva. T viene abortita (rollback).
2. **Se $write_TS(X) \leq TS(T)$:** read(X) è eseguita;
(e di conseguenza il read timestamp è sovrascritto --> $read_TS(X) := TS(T)$)

Esempio:

T_1	T_2	$read_TS(X)$	$write_TS(X)$
	$read(X)$	100	
$read(X)$		110	
	$X := X + 5$		
$X := X + 10$			
$write(X)$			110
	$write(X)$	$read_TS(X) > TS(T_2)$ roll-back di T_2	

Perché questo algoritmo funziona?

1. **Mantiene l'ordine temporale:** L'algoritmo garantisce che le transazioni vengano eseguite in un ordine coerente con i loro timestamp. Se una transazione T1 ha un timestamp inferiore a T2, T1 non può interferire con le operazioni di T2.
2. **Previene le letture sporche (dirty reads):** Una transazione non può leggere un valore scritto da una transazione più recente (condizione $writeTS(X) > TS(T) \rightarrow writeTS(X) > TS(T)$).
3. **Previene le scritture non serializzabili:** Una transazione non può sovrascrivere un valore letto da una transazione più recente (condizione $readTS(X) > TS(T) \rightarrow readTS(X) > TS(T)$).

Esempio:

T1	T2	T3
	READ(X)	
		READ(Y)
	$X = X - 20$	
READ(X)		
$X = X + 10$		
		$Y = Y - 5$
WRITE(X)		
		WRITE(Y)
	WRITE(X)	
READ(Y)		
$Y = Y + 20$		
WRITE(Y)		

Il controllo della concorrenza è basato su timestamp, e le transazioni hanno i seguenti timestamp:

$TS(T1) = 110$, $TS(T2) = 100$, $TS(T3) = 105$.

Assumiamo che all'inizio:

- $RTS(X) = 0$
- $RTS(Y) = 0$
- $WTS(X) = 0$
- $WTS(Y) = 0$

Assumiamo che i valori iniziali degli item X e Y nella memoria condivisa siano rispettivamente x0 e y0, e consideriamo le operazioni dello schedule nell'ordine in cui vengono eseguite.

Passo	Transazione	Operazione	RTS(X)	RTS(Y)	WTS(X)	WTS(Y)	X	Y
1	T2 TS(T2)=100	READ(X)	100	0	0	0	x0	y0
2	T3 TS(T3)=105	READ(Y)	100	105	0	0	x0	y0
3	T2 TS(T2)=100	X=X-20	100	105	0	0	x0	y0
4	T1 TS(T1)=110	READ(X)	110	105	0	0	x0	y0
5	T1 TS(T1)=110	X=X+10	110	105	0	0	x0	y0
6	T3 TS(T3)=105	Y=Y-5	110	105	0	0	x0	y0
7	T1 TS(T1)=110	WRITE(X)	110	105	110	0	x0+10	y0
8	T3 TS(T3)=105	WRITE(Y)	110	105	110	105	x0+10	y0-5
9	T2 TS(T2)=100	WRITE(X) ROLL BACK	110	105	110	105	x0+10	y0-5
10	T1 TS(T1)=110	READ(Y)	110	110	110	105	x0+10	y0-5
11	T1 TS(T1)=110	Y=Y+20	110	110	110	105	x0+10	y0-5
12	T1 TS(T1)=110	WRITE(Y)	110	110	110	110	x0+10	(y0-5)+20

Al passo 9 la transazione T2 viene abortita. Dovrebbe eseguire la scrittura dell'item X, ma il suo timestamp è **minore** del timestamp della transazione **più giovane (con timestamp più alto)** che ha **letto** l'item X ($RTS(X) = 110 > TS(T2) = 100$). Ciò significa che una transazione che **ha iniziato le** proprie operazioni **dopo T2 ha già letto** il valore dell'item X, mentre **secondo l'ordine di esecuzione** avrebbe dovuto leggere il valore di X **già modificato da T2**. Da qui la necessità di eseguire il roll back della transazione T2.